

Implémentation, ServeurMultiThreade (v7)

Fabrice.Kordon@lip6.fr



Objectif de cette version

2

Traiter la remarque sur la v6!

- Suppression des résultats au fur et à mesure
 - ▶ Par le client
 - ▶ Objectif, économiser la mémoire
 - ▶ Les résultats inutiles sont donc supprimés

Les changements?

- Coté client
- Coté serveur
- Facile!!!



La classe LeServeur

3

```
class LeServeur implements Runnable {
    ...
    LeServeur (ArrayBlockingQueue<Requete> fr,
               ExecutorService pc, ArrayList<Future<Requete>> r) {...}
    private void trace (String m) {...}
    public void run() {
        Requete r = null;
        this.trace("initialisé");
        while (!poolClients.isTerminated()) {
            try {
                r = f.poll(1, TimeUnit.SECONDS); // Semi-passif nécessaire
                if (r != null) {
                    this.trace("je traite " + r);
                    Future<Requete> rq = executor.submit(new Servant(r));
                    res.add(rq);
                } else {
                    this.trace("pool terminé = " + poolClients.isTerminated());
                }
            } catch (InterruptedException e) {
                System.err.println ("Serveur, ne doit jamais arriver, " + e);
                return;
            }
        }
        groupeServants.list();
        executor.shutdown();
        //res.clear(); // Nettoyons ces résultats consommés
        this.trace("Adios");
    }
}
```


La classe UnClient

4

```
class UnClient implements Runnable {
    ...
    public void run() {
        this.trace("initialisé");
        Requete r = new Requete(this, generateur.nextInt(1000));
        this.trace("j'envoie la requête " + r);
        try {
            f.put (r);
            this.trace("j'attends le serveur");
            synchronized (veille) {
                veille.wait();
            }
        } catch (InterruptedException e) {
            System.out.println ("UnClient "+monId+", cela doit jamais arriver, " + e);
        }
        this.trace("ma requête a été traitée");
        for (Future<Requete> futReq : res) { // parcourir une collection
            try{
                if (futReq.get().emetteur == this) {
                    this.trace("sa valeur est " + futReq.get().valeur);
                    res.remove(futReq); // Nettoyons au plus tôt
                }
            }
            catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}
```


Une exécution...

5

```
$ java ServeurMultiThreade7 2 3
Serveur : initialisé
UnClient 1 : initialisé
UnClient 2 : initialisé
UnClient 3 : initialisé
java.lang.ThreadGroup[name=serveur,maxpri=10]
  Thread[Thread-0,5,serveur]
java.lang.ThreadGroup[name=clients,maxpri=10]
  Thread[Thread-1,5,clients]
  Thread[Thread-2,5,clients]
    UnClient 1 : j'envoie la requête (client 1, 424)
UnClient 2 : j'envoie la requête (client 2, 805)
UnClient 3 : j'envoie la requête (client 3, 846)
Thread[Thread-3,5,clients]
UnClient 1 : j'attends le serveur
Serveur : je traite (client 1, 424)
UnClient 2 : j'attends le serveur
UnClient 3 : j'attends le serveur
Serveur : je traite (client 2, 805)
Servant (client 1, 424) : initialisé
Serveur : je traite (client 3, 846)
Servant (client 2, 805) : initialisé
Servant (client 3, 846) : initialisé
Servant (client 1, 424) : je prévient le client
UnClient 1 : ma requête a été traitée
UnClient 1 : sa valeur est 424
```


Une exécution...

5

```
Exception in thread "Thread-1" java.util.ConcurrentModificationException
  at java.util.ArrayList$Itr.checkForComodification(ArrayList.java:901)
  at java.util.ArrayList$Itr.next(ArrayList.java:851)
  at UnClient.run(ServeurMultiThreade7.java:161)
  at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1142)
  at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:617)
  at java.lang.Thread.run(Thread.java:745)
```



Que se passe-t-il?

ArrayList n'est pas ThreadSafe



```
Servant (client 2) : ma valeur est 846
UnClient 2 : ma valeur est 846
UnClient 2 : sa valeur est 846
Servant (client 3) : ma valeur est 846
UnClient 3 : ma valeur est 846
UnClient 3 : sa valeur est 846
```

```
Exception in thread "Thread-3" java.util.ConcurrentModificationException
  at java.util.ArrayList$Itr.checkForComodification(ArrayList.java:901)
  at java.util.ArrayList$Itr.next(ArrayList.java:851)
  at UnClient.run(ServeurMultiThreade7.java:161)
  at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1142)
  at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:617)
  at java.lang.Thread.run(Thread.java:745)
```

```
Serveur : pool terminé = true
java.lang.ThreadGroup[name=servants,maxpri=10]
  Thread[Thread-4,5,servants]
  Thread[Thread-5,5,servants]
  Thread[Thread-6,5,servants]
```

Serveur : Adios

Au travail!!!

6

Que mettre dans la nouvelle version?

-  Choisir la bonne collection...
-  Utilisation de
 - ▶ `java.util.concurrent.CopyOnWriteArrayList`
-  Au lieu de
 - ▶ `import java.util.ArrayList`

Remplacement systématique...

-  Souvenez-vous...
 - ▶ `CopyOnWriteArrayList` est la version «thread safe» de `ArrayList`



Exécution de la v7bis...

7

```
$ java ServeurMultiThreade7bis 2 3
Serveur : initialisé
UnClient 1 : initialisé
UnClient 2 : initialisé
java.lang.ThreadGroup[name=serveur,maxpri=10]
  UnClient 3 : initialisé
Thread[Thread-0,5,serveur]
java.lang.ThreadGroup[name=clients,maxpri=10]
  Thread[Thread-1,5,clients]
    UnClient 3 : j'envoie la requête (client 3, 351)
    UnClient 2 : j'envoie la requête (client 2, 793)
UnClient 1 : j'envoie la requête (client 1, 784)
Thread[Thread-2,5,clients]
  UnClient 3 : j'attends le serveur
UnClient 2 : j'attends le serveur
Thread[Thread-3,5,clients]
  Serveur : je traite (client 3, 351)
UnClient 1 : j'attends le serveur
  Serveur : je traite (client 2, 793)
  Servant (client 3, 351) : initialisé
  Serveur : je traite (client 1, 784)
  Servant (client 2, 793) : initialisé
  Servant (client 1, 784) : initialisé
  Servant (client 3, 351) : je prévient le client
UnClient 3 : ma requête a été traitée
UnClient 3 : sa valeur est 351
  Servant (client 1, 784) : je prévient le client
UnClient 1 : ma requête a été traitée
  Servant (client 2, 793) : je prévient le client
```

```
UnClient 2 : ma requête a été traitée
UnClient 1 : sa valeur est 784
UnClient 2 : sa valeur est 793
Serveur : pool terminé = true
java.lang.ThreadGroup[name=servants,maxpri=10]
  Thread[Thread-4,5,servants]
  Thread[Thread-5,5,servants]
  Thread[Thread-6,5,servants]
Serveur : Adios
```



Et la!!!
Tout va bien